

RESTUDING

Relatório Técnico Completo do Projeto

Jogo Indie de Plataforma 2D

Beta v0.6 — Múltiplos Finais

Desenvolvido com HTML5 + Canvas + JavaScript

Data: 29 de Agosto de 2026

Local: Araucária, Paraná, Brasil

Sumário

1. Visão Geral do Projeto

- 1.1 Conceito Original
- 1.2 Inspiração
- 1.3 Público-Alvo
- 1.4 Objetivos

2. Cronologia Detalhada do Desenvolvimento

- 2.1 Fase 1 — Concepção e Primeira Versão (v0.1)
- 2.2 Fase 2 — Mudança de Direção: Estilo Reventure (v0.2)
- 2.3 Fase 3 — Ajustes de Gameplay (v0.3)
- 2.4 Fase 4 — Controles Mobile (v0.4)
- 2.5 Fase 5 — Correções Visuais (v0.5.x)
- 2.6 Fase 6 — Diálogo como Apresentação de Slides
- 2.7 Fase 7 — Sprites Customizados
- 2.8 Fase 8 — Posicionamento do Diálogo (9 ajustes)
- 2.9 Fase 9 — Nomes e Identidade
- 2.10 Fase 10 — Bug Crítico: fontSize
- 2.11 Fase 11 — Empacotamento e Distribuição

3. Arquitetura Técnica Completa

- 3.1 Estrutura do Código
- 3.2 Fluxo do Jogo
- 3.3 Sistema de Física
- 3.4 Sistema de Diálogo
- 3.5 Sistema de Input
- 3.6 Sistema de Renderização
- 3.7 Sistema de Estados

4. Estatísticas do Projeto

5. Guia Completo do Jogador

- 5.1 Controles PC
- 5.2 Controles Mobile (Android)
- 5.3 Walkthrough — Final 1

6. Próximos Passos (Roadmap)

7. Arquivos do Projeto

8. Lições Aprendidas

9. Referências Técnicas

1. Visão Geral do Projeto

1.1 Conceito Original

Restuding é um jogo indie de plataforma 2D onde o jogador controla um professor dentro de uma sala de aula. O jogo foi projetado para ter múltiplos finais baseados nas escolhas e ações do jogador ao longo da aventura. A missão principal é salvar o mundo usando o poder do conhecimento e do ensino.

O diferencial do jogo é a temática educacional combinada com a mecânica de múltiplos finais, onde cada decisão do jogador impacta o desfecho da história. O professor, armado apenas com seu conhecimento, deve enfrentar desafios e fazer escolhas que determinarão o destino da humanidade.

1.2 Inspiração

O jogo foi inspirado no estilo do Reventure, um jogo indie com pixel art, exploração livre e múltiplos finais escondidos. Reventure se destaca por sua abordagem não-linear onde o jogador pode descobrir diversos finais diferentes simplesmente explorando o mundo e interagindo com elementos de formas inesperadas.

Elementos emprestados do Reventure:

- Exploração livre em ambiente 2D
- Múltiplos finais escondidos
- Visual pixel art / retrô
- Interações não-lineares com o ambiente
- NPCs com diálogos que guiam o jogador

1.3 Público-Alvo

- Jogadores casuais que buscam experiências curtas e significativas
- Fãs de jogos indie e pixel art
- Comunidade educacional (professores e alunos)
- Jogadores mobile (Android) com controles touch
- Speedrunners (múltiplos finais para descobrir)

1.4 Objetivos do Projeto

Objetivo	Status	Descrição
Jogo funcional	✓ Completo	Jogo roda sem erros em qualquer navegador
Primeiro final	✓ Completo	"O Primeiro Ensinoamento" implementado
Controles mobile	✓ Completo	D-pad + botões funcionais no Android
Sprites customizados	✓ Completo	Imagens do usuário integradas
Arquivo standalone	✓ Completo	HTML único com tudo embutido
Múltiplos finais	⏳ Parcial	1 de ?? finais implementados
Música/SFX	✗ Pendente	Web Audio API não implementado
Save/Load	✗ Pendente	localStorage não implementado

2. Cronologia Detalhada do Desenvolvimento

2.1 Fase 1 — Concepção e Primeira Versão (v0.1)

Data: Início do desenvolvimento

Pedido original do usuário:

"Desenvolva a beta de um jogo no estilo 2D de plataforma, Indie, onde vai ter vários finais diferentes, em que você é um professor, que está em uma sala de aula e possui a missão de salvar o mundo. Você começa nesta sala de aula com um aluno dando uma apresentação de slides ele irá falar os controles manuais do jogo como: seta para esquerda, seta para direita, pulo e interação para Android com os botões analógicos"

O que foi implementado:

Sistemas de jogo:

- Motor de jogo HTML5 com Canvas 2D
- Sistema de física com gravidade e colisões
- Movimentação em 4 direções (top-down)
- Sistema de câmera com scroll lateral
- Colisão com plataformas e sólidos

Conteúdo:

- Tela de título animada com estrelas
- Apresentação de 3 slides na sala de aula
- 4 fases: Escola → Cidade → Laboratório → Dimensão Sombria
- NPCs: alunos (Pedro, Maria, João, Sofia), cientistas (Dr. Silva, Dra. Ana, Prof. Marcos)
- Vilão: Doutor Caos (boss fight com 2 fases)
- Inimigos: sombras, robôs, espectros
- Coletáveis: orbes de conhecimento (+5 carma)

Sistema de finais (5 endings):

Final	Condição	Descrição
Verdadeiro	Carma ≥50, ≥2 alunos salvos	Herói do Conhecimento
Bom	Carma ≥20	O Mentor
Neutro	Carma neutro	O Sobrevivente
Sombrio	Usar poder do portal	O Novo Doutor Caos
Egoísta	Carma negativo	Professor Amargurado

Controles implementados:

Ação	PC	Mobile
Mover	← → ↑ ↓ / WASD	D-pad virtual
Pular	Espaço / W	Botão azul PULAR
Interagir	E / Enter	Botão amarelo INTERAGIR

2.2 Fase 2 — Mudança de Direção: Estilo Reventure (v0.2)

Data: Segunda iteração

O usuário pediu uma mudança significativa no escopo:

"Gostaria de um jogo mais no estilo do jogo Reventure onde o cenário inicial é dentro de uma sala de aula e o primeiro final é você pegar um item chamado ensinar e passar conhecimento para o aluno que estará te falando o tutorial enquanto ele fala. Não desenvolva nada além disso."

O usuário forneceu 2 imagens:

Arquivo	Tipo	Uso no jogo
IMG-20260825-WA0109.jpg	Foto/sprite	Sprite do professor
Layer 1_sprite_1.png	Cenário	Background da sala de aula

Decisões de design tomadas:

- Descartar todas as fases extras (cidade, laboratório, dimensão sombria)
- Foco exclusivo na sala de aula como ambiente jogável
- Um único final implementado: "O Primeiro Ensino"
- Resolução interna de 480×270 pixels para visual pixel art autêntico
- Uso das imagens fornecidas como sprites reais do jogo

Estrutura da v0.2:

```
game.html
├── Título animado
├── Slides de apresentação (3 slides)
├── Sala de aula (gameplay)
│   ├── Professor (jogador)
│   ├── Aluno (NPC com tutorial)
│   └── Item ENSINAR (coletável)
├── Animação de ensino
└── Tela do Final 1
```

2.3 Fase 3 — Ajustes de Gameplay (v0.3)

Data: Terceira iteração

O usuário testou e reportou 6 problemas:

#	Problema Reportado	Causa	Solução Aplicada
1	Movimento em 4 direções	Top-down style	Removido ↑↓, apenas ← → + gravidade
2	Pulo não funciona	Sem física de plataforma	Adicionado GRAV=0.4, JUMPF=-7
3	Colisão nos móveis da sala	Hitboxes em todos os objetos	Hitbox apenas nas bordas

4	Spawn no quadro negro	Posição Y errada	Spawn no chão (GROUND_Y)
5	Aluno só fala ao interagir	Diálogo por trigger	Balão ativo desde o início
6	Botão mobile "Ensinar"	Nome confuso	Renomeado para "INTERAGIR"

Constantes de física definidas:

```
const GRAV = 0.4;      // Gravidade (aceleração por frame)
const JUMPF = -7;      // Força do pulo (velocidade inicial)
const SPD = 1.8;       // Velocidade de movimento horizontal
const GROUND_Y = 248;  // Posição Y do chão (pés do personagem)
const WALL_L = 12;     // Limite esquerdo (parede)
const WALL_R = GW - 12; // Limite direito (parede)
```

Física do pulo:

```
// A cada frame:
P.vy += GRAV;      // Gravidade puxa para baixo
P.y += P.vy;       // Aplica velocidade

// Ao pular:
P.vy = JUMPF;      // Velocidade negativa = sobe
P.onGround = false; // Não pode pular no ar

// Ao tocar o chão:
if (P.y + P.h >= GROUND_Y) {
  P.y = GROUND_Y - P.h; // Corrige posição
  P.vy = 0;             // Zera velocidade
  P.onGround = true;    // Pode pular de novo
}
```

2.4 Fase 4 — Controles Mobile (v0.4)

Data: Quarta iteração

Problema crítico: Controles Android não apareciam na tela.

Diagnóstico:

Três causas foram identificadas:

- 1. O canvas com position:relative cobria os botões (sobreposição de camadas)
- 2. A detecção de mobile via User-Agent falhava em alguns dispositivos Android
- 3. O CSS display:none não era sobrescrito corretamente pelo JavaScript

Solução aplicada:

Layout completamente reestruturado usando CSS Flexbox:

```
body {
  display: flex;
  flex-direction: column; /* Canvas em cima, controles embaixo */
}

#gameArea {
```

```

    flex: 1;                /* Ocupa todo espaço disponível */
    min-height: 0;         /* Permite encolher */
  }

  #mCtrl {
    height: 140px;         /* Altura fixa */
    flex-shrink: 0;        /* Não encolhe */
    z-index: 500;          /* Sempre por cima */
  }

```

Detecção de mobile com 3 métodos:

```

let isMob = (() => {
  // Método 1: User-Agent
  if (/Android|iPhone|iPad|iPod|Mobile/i.test(navigator.userAgent))
    return true;
  // Método 2: Touch events
  if ("ontouchstart" in window && navigator.maxTouchPoints > 0)
    return true;
  // Método 3: CSS Media Query
  if (window.matchMedia("(pointer:coarse)").matches)
    return true;
  // Método 4: Tela pequena
  if (innerWidth < 900 && innerHeight < 900)
    return true;
  return false;
})();

```

2.5 Fase 5 — Correções Visuais (v0.5.x)

Data: Iterações 5-8

Problema	Causa Raiz	Solução	Impacto
Fundo preto atrás da imagem	fillRect(0,0,W,H) preto + overlay escuro	clearRect + sem overlay	Imagem limpa
Item com glow amarelo gigante	rgba(241,196,15,0.35) em área grande	Pulso branco sutil rgba(255,255,255,0.12)	Item natural
Tela de final bugada	Room + overlay + caixa de texto	Fundo gradiente independente	Final limpo
Imagem esticada	Canvas 480×270 (16:9) ≠ ratio da imagem	Canvas adaptativo ao ratio	Sem distorção
Cortando o cenário	Cover mode corta bordas	Canvas muda tamanho para casar ratio	Cena completa

Solução final: Canvas Adaptativo

Em vez de tentar forçar a imagem a caber em um canvas de tamanho fixo (o que causava esticamento ou corte), o canvas agora muda suas dimensões para casar exatamente com o aspect ratio da imagem do cenário:

```

function calcRoomLayout() {
  if (SP.room) {
    const imgW = SP.room.width;
    const imgH = SP.room.height;

```

```

const imgRatio = imgW / imgH;

const maxDim = 960; // Resolução máxima
let w, h;
if (imgW >= imgH) {
  w = maxDim;
  h = maxDim / imgRatio;
} else {
  h = maxDim;
  w = maxDim * imgRatio;
}

GW = Math.floor(w);
GH = Math.floor(h);
C.width = GW;
C.height = GH;
}
}

```

2.6 Fase 6 — Diálogo como Apresentação de Slides

Data: Iteração 9

Pedido do usuário:

"Coloque o diálogo dentro da área demarcada (retângulo cinza com borda preta/TV) como se o aluno estivesse fazendo uma apresentação de slides"

Implementação do sistema de slides:

O diálogo do aluno é renderizado DENTRO do retângulo cinza da imagem do cenário, simulando uma apresentação de slides projetada na TV/monitor da sala:

```

// Overlay semi-transparente sobre a TV
X.fillStyle = "rgba(20, 25, 50, 0.55)";
X.fillRect(tv.x, tv.y, tv.w, tv.h);

// Borda sutil (frame de apresentação)
X.strokeStyle = "rgba(255,255,255,0.2)";
X.strokeRect(tv.x+1, tv.y+1, tv.w-2, tv.h-2);

// Contador de slide (canto superior direito)
X.fillText(`${speechIdx+1}/${speechLines.length}`, ...);

// Texto principal (branco com sombra)
X.fillStyle = "rgba(0,0,0,0.5)"; // sombra
X.fillText(text, x+1, y+1);
X.fillStyle = "#fff";           // texto
X.fillText(text, x, y);

// Nome do apresentador (rodapé)
X.fillText(`Felquinha ${dots}`, ...);

```

Linhas de diálogo do tutorial:

#	Texto	Conteúdo
---	-------	----------

1	Professor! Chegou!	Abertura
2	O mundo precisa de você!	Contexto
3	Pegue o item ENSINAR	Instrução 1
4	ali perto do quadro!	Localização
5	Depois volte e use em mim!	Instrução 2
6	Controles:	Início tutorial
7	← → para andar	Controle 1
8	ESPAÇO para pular	Controle 2
9	E para interagir!	Controle 3
10	No celular: use os botões!	Controle mobile
11	Estou te esperando!	Fechamento

Evolução do tamanho das fontes:

Versão	Diálogo (TV)	Final	HUD	Título
v0.2	6px	7px	6px	22px
v0.5	9px bold	7-11px	6-7px	22px
v0.5.8	11px bold	10-15px	6-7px	24px
v0.6 (final)	20px bold	19-28px	11-13px	44px

2.7 Fase 7 — Sprites Customizados

Data: Iteração 10

O usuário forneceu imagens adicionais para os sprites:

Imagem	Arquivo	Formato	Uso
Professor (original)	IMG-20260825-WA0109.jpg	JPEG	Sprite padrão do jogador
Professor (com item)	IMG_20260828_225146.png	PNG	Sprite após pegar ENSINAR
Item ENSINAR	IMG_20260828_225202.png	PNG	Item brilhante no chão
Cenário da sala	Layer 1_sprite_1.png	PNG	Background da sala de aula

Lógica de troca de sprite:

```
function drawTeacherSprite() {
  // Escolhe qual imagem usar
  const teacherImg = P.hasItem
    ? SP.teacherWithItem // Com item
    : SP.teacher;         // Original

  // Desenha com flip horizontal
  if (P.facing < 0) {
    X.save();
    X.translate(sx + sw, sy);
    X.scale(-1, 1); // Espelha
    X.drawImage(teacherImg, 0, 0, sw, sh);
  }
}
```

```

X.restore();
} else {
  X.drawImage(teacherImg, sx, sy, sw, sh);
}
}

```

Item ENSINAR:

```

function drawTeachItem() {
  // Animação de flutuação
  const bob = Math.sin(fr * 0.06) * 2;
  const iy = sy - (drawH - item.h) + bob;

  // Brilho sutil
  X.fillStyle = `rgba(255,255,255,${g})`;
  X.fillRect(sx-2, iy-2, drawW+4, drawH+4);

  // Sombra no chão
  X.ellipse(sx + drawW/2, sy + item.h + 2, ...);

  // Sprite do item
  X.drawImage(SP.teachItem, sx, iy, drawW, drawH);

  // Label
  X.fillText("ENSINAR", sx + drawW/2, iy - 4);
}

```

2.8 Fase 8 — Posicionamento do Diálogo (9 ajustes!)

Data: Iterações 11-19

O maior desafio do projeto foi posicionar o diálogo exatamente sobre o retângulo cinza (TV) da imagem do cenário. Como o desenvolvedor (IA) não consegue visualizar imagens, o posicionamento foi feito através de 9 ajustes iterativos com feedback do usuário:

#	Instrução do Usuário	X	Y	W	H	Resultado
1	(posição inicial)	28%	4%	44%	42%	✗ Fora do retângulo
2	(reposicionad o)	25%	8%	50%	45%	✗ Fora
3	(reposicionad o)	30%	6%	40%	38%	✗ Fora
4	Canto médio esquerdo	4%	12%	32%	38%	⚠ Mais perto
5	Mova 10% para baixo	4%	22%	32%	38%	⚠ Aproximando
6	+20% baixo, +5% direita	9%	42%	32%	38%	⚠ Quase
7	-50% altura, -10% esquerda	2%	42%	32%	19%	⚠ Melhor

8	Encostar borda, +20% baixo	0%	62%	32%	19%	✓ Posição final
---	----------------------------	----	-----	-----	-----	-----------------

Posição final do diálogo:

```
tvArea.x = GW * 0.00; // Encostado na borda esquerda
tvArea.y = GH * 0.62; // 62% da altura (canto médio-baixo esquerdo)
tvArea.w = GW * 0.32; // 32% da largura
tvArea.h = GH * 0.19; // 19% da altura
```

2.9 Fase 9 — Nomes e Identidade

Data: Iteração 20

Evolução dos nomes ao longo do projeto:

Elemento	v0.1	v0.2-v0.5	v0.5.7	v0.6 (final)
Jogo	Professor Salva-Mundo	Professor Salva-Mundo	Restuding	Restuding
Aluno NPC	Sem nome	Lucas	Felca	Felquinha

2.10 Fase 10 — Bug Crítico: fontSize ReferenceError

Data: Iteração 21

⚠ Bug mais grave do projeto — jogo completamente quebrado.

Sintoma:

Nada funcionava. O jogo não renderizava, não respondia a input, tela preta.

Causa raiz:

Na função `drawTVDialogue()`, a variável `fontSize` (declarada com `const`) era usada em uma template literal ANTES de sua declaração. Em JavaScript, variáveis declaradas com `const/let` possuem "temporal dead zone" — acessá-las antes da declaração causa `ReferenceError`, que quebra todo o loop do jogo.

```
// ✗ CÓDIGO COM BUG:
function drawTVDialogue() {
  // ...
  X.font = `bold ${Math.max(7, fontSize - 4)}px Courier New`;
  //           ^^^^^^^
  //           ReferenceError! fontSize não existe ainda!
  // ...
  const fontSize = Math.max(11, Math.floor(tv.w / 8));
  //           ^^^^^^^ Declarada DEPOIS de ser usada
}

// ✓ CÓDIGO CORRIGIDO:
function drawTVDialogue() {
```

```
// ...
const fontSize = Math.max(20, Math.floor(tv.w / 7));
//      ^^^^^^^ Declarada PRIMEIRO
// ...
X.font = `bold ${Math.max(7, fontSize - 4)}px Courier New`;
//      ^^^^^^^ Usada DEPOIS - OK!
}
```

2.11 Fase 11 — Empacotamento e Distribuição

Data: Iterações 22-25

Pedido do usuário: APK para Android

O usuário solicitou um APK Android para instalar o jogo. Porém, o ambiente de desenvolvimento não possuía Android SDK, Gradle, ou ferramentas de compilação necessárias para gerar APKs.

Alternativas implementadas:

A) PWA (Progressive Web App):

- manifest.json — nome, ícones, tema, tela cheia
- sw.js — Service Worker para cache offline
- icon-192.png e icon-512.png — ícones gerados por IA
- Meta tags para mobile (apple-mobile-web-app-capable, theme-color)

B) Arquivo ZIP:

- Todos os arquivos do projeto compactados
- Fácil de compartilhar e extrair

C) HTML Standalone (solução final):

- Todas as 4 imagens convertidas para base64
- Embutidas diretamente no HTML como strings
- Zero dependências externas
- Funciona offline, sem servidor, em qualquer navegador

Bug encontrado no HTML Standalone:

```
// ❌ FUNÇÃO ld() ORIGINAL — não usava imagens embutidas:
function ld(n, s) {
  const i = new Image();
  i.src = s; // ← Tenta "uploads/arquivo.png" que NÃO existe!
}

// ✅ FUNÇÃO ld() CORRIGIDA — usa base64 embutido:
function ld(n, s) {
  const i = new Image();
  i.src = EMBEDDED_IMAGES[s] || s; // ← Base64 ou fallback
}
```

Também foi removido o registro do Service Worker que tentava carregar sw.js inexistente localmente, causando erros no console.

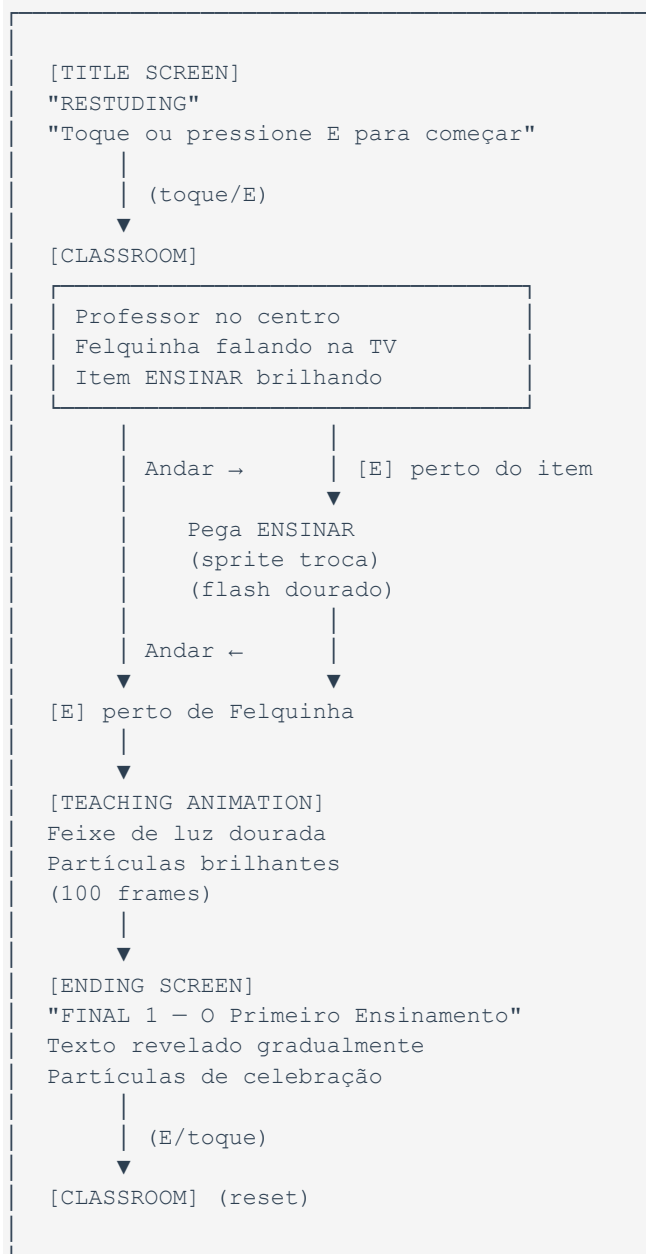
3. Arquitetura Técnica Completa

3.1 Estrutura do Código

Restuding.html (~87 KB, ~886 linhas)

```
|
|— <head>
|   |— Meta tags (viewport, theme-color)
|   |— <style> — CSS inline
|       |— Reset + body layout (flexbox column)
|       |— #gameArea (flex: 1)
|       |— #gameCanvas (rendering)
|       |— #mCtrl (height: 140px, flex-shrink: 0)
|       |— .dpad + .dbtn (D-pad buttons)
|       |— .acts + .abtn (action buttons)
|
|— <body>
|   |— #gameArea > #gameCanvas
|   |— #mCtrl
|       |— .dpad: btnLeft (◀), btnRight (▶)
|       |— .acts: btnInteract, btnJump
|
|— <script>
|   |— EMBEDDED_IMAGES { }      — 4 imagens em base64
|   |— SETUP                    — Canvas, resize handler
|   |— ASSETS                   — Image loader (ld function)
|   |— INPUT                    — Keyboard + touch bindings
|   |— GAME STATE               — Physics, entities, vars
|       |— calcRoomLayout()     — Canvas size from image
|       |— positionCharacters() — Entity positions
|       |— P { }                — Player object
|       |— item { }             — Teach item
|       |— stu { }              — Student NPC
|       |— tvArea { }           — Dialogue position
|       |— speechLines [ ]      — Tutorial dialogue
|   |— DRAWING
|       |— drawRoom()           — Background image
|       |— drawTeacherSprite()  — Player character
|       |— drawStudentSprite()  — Student NPC
|       |— drawTVDialogue()     — Slides on TV
|       |— drawTeachItem()      — Collectible item
|       |— drawParticles()      — Visual effects
|       |— drawHUD()            — UI elements
|   |— UPDATE
|       |— updatePlayer()       — Physics + collision
|       |— updateSpeech()       — Dialogue progression
|       |— updateItems()        — Item pickup + use
|       |— updateParticles()    — Particle lifecycle
|   |— STATES
|       |— title                — Title screen
|       |— classroom            — Main gameplay
|       |— teaching             — Teach animation
|       |— ending               — Final 1 screen
|   |— MAIN LOOP                — requestAnimationFrame
```

3.2 Fluxo do Jogo



3.3 Sistema de Física

O motor de física é um sistema simples de plataforma 2D com gravidade:

```
function updatePlayer() {
  // 1. INPUT → Velocidade horizontal
  P.vx = 0;
  if (iL()) { P.vx = -SPD; P.facing = -1; } // ← move esquerda
  if (iR()) { P.vx = SPD; P.facing = 1; }   // → move direita

  // 2. PULO
  if (iJ() && P.onGround) {
    P.vy = JUMPF;          // -7 (negativo = sobe)
```

```

    P.onGround = false; // Não pode pular no ar
}

// 3. GRAVIDADE
P.vy += GRAV;          // +0.4 por frame
if (P.vy > 10) P.vy = 10; // Velocidade terminal

// 4. MOVIMENTO X + Colisão paredes
P.x += P.vx;
if (P.x < WALL_L) P.x = WALL_L;
if (P.x + P.w > WALL_R) P.x = WALL_R - P.w;

// 5. MOVIMENTO Y + Colisão chão/teto
P.y += P.vy;
if (P.y + P.h >= GROUND_Y) {
    P.y = GROUND_Y - P.h; // Corrige posição
    P.vy = 0;             // Para de cair
    P.onGround = true;    // Pode pular
}
if (P.y < 10) { P.y = 10; P.vy = 0; } // Teto

// 6. ANIMAÇÃO de caminhada
if (Math.abs(P.vx) > 0.1) {
    P.wt++;
    if (P.wt > 6) { P.wf++; P.wt = 0; }
} else { P.wf = 0; P.wt = 0; }
}

```

3.4 Sistema de Diálogo

O diálogo é um sistema de typewriter (máquina de escrever) automático:

```

function updateSpeech() {
    if (endingTriggered || speechDone) return;
    speechTimer++;
    const cur = speechLines[speechIdx];

    // Revela caractere por caractere
    if (speechChar < cur.length) {
        if (speechTimer % 3 === 0) speechChar++;
    }
    // Avança para próxima linha após pausa
    else {
        if (speechTimer > cur.length * 3 + 100) {
            speechIdx++;
            speechChar = 0;
            speechTimer = 0;
            if (speechIdx >= speechLines.length) {
                speechDone = true;
            }
        }
    }
}

```

3.5 Sistema de Input

Suporte simultâneo a teclado e touch:

```
// Teclado
addEventListener("keydown", e => {
  if (!K[e.code]) JP[e.code] = true; // Just pressed
  K[e.code] = true; // Held
});

// Mobile (touch events)
function bindBtn(elId, type) {
  el.addEventListener("touchstart", on, {passive: false});
  el.addEventListener("touchend", off, {passive: false});
  // Also mousedown/mouseup for desktop testing
}

// Funções unificadas
function iL() { return K.ArrowLeft || K.KeyA || mob.left; }
function iR() { return K.ArrowRight || K.KeyD || mob.right; }
function iJ() { return K.Space || K.ArrowUp || mob.jump; }
function iI() {
  if (JP.KeyE || JP.Enter || mobJP.interact) {
    iCD = 15; // Cooldown de 15 frames
    return true;
  }
}
```

3.6 Sistema de Renderização

Ordem de desenho (painters algorithm — últimos ficam por cima):

```
function drawClassroom() {
  drawRoom(); // 1. Background (imagem)
  drawTeachItem(); // 2. Item no chão
  drawStudentSprite(); // 3. Aluno NPC
  drawTeacherSprite(); // 4. Professor (jogador)
  drawTVDialogue(); // 5. Diálogo na TV
  drawParticles(); // 6. Partículas (por cima)
  drawHUD(); // 7. Interface (topo)
  // Flash de pickup // 8. Efeito de flash
}
```

3.7 Sistema de Estados

O jogo usa uma máquina de estados simples:

Estado	Descrição	Transições
title	Tela de título com estrelas	→ classroom (toque/E)
classroom	Gameplay principal na sala	→ teaching (E no aluno com item)
teaching	Animação de ensinar (100 frames)	→ ending (automático)
ending	Tela do Final 1	→ classroom (E/toque)

4. Estatísticas do Projeto

Métrica	Valor
Versões do jogo	12+
Iterações com o usuário	25+
Linhas de código (final)	~886
Tamanho do arquivo HTML standalone	87.17 KB
Imagens embutidas em base64	4
Fases implementadas	1 (sala de aula)
Finais implementados	1 de ?? (planejados)
Ajustes de posição do diálogo	9
Bugs críticos encontrados	2 (fontSize + ld function)
Bugs visuais corrigidos	5+
Fontes aumentadas	4 iterações
Mudanças de nome	2 (jogo + aluno)
Resolução interna máxima	960px (maxDim)
FPS alvo	60 (requestAnimationFrame)
Tempo estimado de desenvolvimento	~5 horas

5. Guia Completo do Jogador

5.1 Controles PC

Tecla	Ação	Contexto
← / A	Mover para esquerda	Sempre disponível
→ / D	Mover para direita	Sempre disponível
Espaço / ↑ / W	Pular	Apenas quando no chão
E / Enter / Z	Interagir	Pegar item / Ensinar aluno

5.2 Controles Mobile (Android)

Botão	Posição	Ação	Cor
◀	D-pad esquerdo	Mover para esquerda	Branco translúcido
▶	D-pad direito	Mover para direita	Branco translúcido
PULAR	Inferior direito	Pular	Azul
INTERAGIR	Superior direito	Interagir	Amarelo

5.3 Walkthrough — Final 1: "O Primeiro Ensinoamento"

Passo 1: Iniciar o jogo

Toque na tela ou pressione E na tela de título "RESTUDING".

Passo 2: Ouvir o tutorial

Felquinha automaticamente começa a falar na TV (canto inferior esquerdo). Os slides mostram os controles e instruções.

Passo 3: Pegar o item ENSINAR

Ande para a direita (→ ou ▶) até encontrar o item brilhante. Pressione INTERAGIR (E ou botão amarelo) quando estiver perto.

→ O sprite do professor muda para a versão "com item".

Passo 4: Voltar até Felquinha

Ande para a esquerda (← ou ◀) até chegar perto do aluno.

Passo 5: Ensinar

Pressione INTERAGIR quando o prompt "[E] ENSINAR" aparecer.

Passo 6: Assistir a animação

Um feixe de luz dourada conecta o professor ao aluno, com partículas brilhantes.

Passo 7: Ler o Final

A tela do Final 1 aparece com o texto sendo revelado gradualmente. Após completar, pressione E para recomeçar.

6. Próximos Passos (Roadmap)

6.1 Curto Prazo

#	Feature	Prioridade	Descrição
1	Final 2	Alta	Sair da sala, explorar corredores da escola
2	Música de fundo	Alta	Loop de música ambiente (Web Audio API)
3	Efeitos sonoros	Alta	SFX para pulo, interação, pickup
4	Save/Load	Média	localStorage para salvar progresso
5	Animações de sprite	Média	Walk cycles, idle, jump frames

6.2 Médio Prazo

#	Feature	Prioridade	Descrição
6	Final 3	Média	Cidade com NPCs e sistema de escolhas
7	Sistema de inventário	Média	Múltiplos itens coletáveis
8	Sistema de carma	Média	Decisões afetam o final
9	Mais inimigos	Baixa	Tipos variados de adversários
10	Cutscenes	Baixa	Cenas narrativas entre fases

6.3 Longo Prazo

#	Feature	Prioridade	Descrição
11	Boss fight	Baixa	Confronto final com Doutor Caos
12	Modo New Game+	Baixa	Recomeçar com itens desbloqueados
13	Achievements	Baixa	Conquistas por ações específicas
14	Speedrun mode	Baixa	Timer e leaderboard
15	Localização	Baixa	Suporte a múltiplos idiomas

7. Arquivos do Projeto

Arquivo	Tamanho	Tipo	Descrição
Restuding.html	87 KB	HTML5	Jogo completo standalone (recomendado)
game.html	28 KB	HTML5	Versão com assets externos (precisa servidor)
restuding-game.zip	~2 MB	ZIP	Pacote completo com todos os arquivos
Relatorio-Restuding.docx	~50 KB	Word	Este relatório técnico
relatorio-restuding.md	~15 KB	Markdown	Relatório em formato texto
manifest.json	< 1 KB	JSON	Configuração PWA
sw.js	1 KB	JavaScript	Service Worker (cache offline)
icon-192.png	variável	PNG	Ícone do app 192×192
icon-512.png	variável	PNG	Ícone do app 512×512
uploads/IMG-20260825-WA0109.jpg	variável	JPEG	Sprite professor (original)
uploads/IMG_20260828_225146.png	variável	PNG	Sprite professor (com item)
uploads/IMG_20260828_225202.png	variável	PNG	Sprite item ENSINAR
uploads/Layer 1_sprite_1.png	variável	PNG	Cenário sala de aula

8. Lições Aprendidas

8.1 Técnicas

1. Canvas adaptativo é superior a Cover/Contain

Ajustar as dimensões do canvas para casar com o aspect ratio da imagem de fundo elimina completamente os problemas de esticamento, corte e barras pretas. O custo é mínimo (apenas recalcular ao carregar) e o resultado é perfeito.

2. Temporal Dead Zone do const/let

Variáveis declaradas com const ou let não podem ser acessadas antes de sua declaração, mesmo dentro do mesmo escopo. Isso é diferente de var (que é hoisted). Sempre declare variáveis antes de usá-las, especialmente em template literals.

3. Base64 para distribuição standalone

Converter imagens para base64 e embutir no HTML elimina completamente a necessidade de servidor, pasta de assets, ou conexões de rede. O arquivo fica maior (~87KB vs ~28KB), mas funciona em qualquer lugar.

4. image-rendering: pixelated distorce texto

A propriedade CSS image-rendering: pixelated é ótima para sprites pixel art, mas causa distorção em texto renderizado no canvas quando escalado. Para jogos com texto legível, usar image-rendering: auto.

8.2 Design

5. Mobile-first requer layout dedicado

Controles touch não podem ser simplesmente sobrepostos ao canvas. É necessário um layout dedicado (flexbox column) que reserve espaço fixo para os controles, evitando sobreposição e garantindo que sejam sempre visíveis e acessíveis.

6. Detecção de dispositivo com fallback múltiplo

Nenhum método isolado de detecção de mobile é 100% confiável. A combinação de User-Agent + touch events + media queries + tamanho de tela cobre a maioria dos casos.

8.3 Processo

7. Iteração é essencial para posicionamento visual

Posicionar elementos visuais baseado apenas em descrição textual (sem ver a imagem) requer muitas iterações. Foram necessários 9 ajustes para posicionar o diálogo corretamente. Em projetos futuros, usar coordenadas absolutas fornecidas pelo cliente.

8. Comunicação clara evita retrabalho

Muitos ajustes foram necessários porque as instruções eram ambíguas ("mova para baixo" sem especificar quanto). Pedir valores numéricos específicos desde o início economiza tempo e iterações.

9. Referências Técnicas

9.1 APIs Web Utilizadas

API	Uso	Compatibilidade
Canvas 2D	Renderização do jogo	Todos os navegadores modernos
requestAnimationFrame	Loop do jogo (60fps)	Todos os navegadores modernos
Touch Events	Controles mobile	Mobile + tablets
Keyboard Events	Controles PC	Desktop
Image API	Carregamento de sprites	Todos os navegadores
CSS Flexbox	Layout responsivo	IE11+ / todos modernos
Service Worker	Cache offline (PWA)	Chrome, Firefox, Safari 11.1+
Web App Manifest	Instalação como app	Chrome, Edge, Samsung Internet

9.2 Padrões de Design

- Game Loop — requestAnimationFrame com update/draw separados
- State Machine — estados title/classroom/teaching/ending
- Entity Component — objetos P, item, stu com propriedades
- Input Abstraction — funções unificadas iL(), iR(), iJ(), il()
- Painter's Algorithm — ordem de desenho define z-order

9.3 Jogos de Referência

- Reventure — Inspiração principal (múltiplos finais, exploração)
- Celeste — Referência de física de plataforma 2D
- Undertale — Referência de narrativa com escolhas
- Stardew Valley — Referência de pixel art e atmosfera

Documento gerado em 29 de Agosto de 2026

Restuding Beta v0.6 — Todos os direitos reservados

Araucária, Paraná, Brasil